

Programación I

Comparar cosas y tomar decisiones

Parte II · 28 de mayo, 2026

Lo que vemos hoy

- 01 Los 6 operadores de comparación
- 02 Guardar el resultado de una comparación
- 03 Diferencia entre `=` y `==`
- 04 `and`, `or`, `not`
- 05 Errores típicos
- 06 El problema de `input()` con números
- 07 Ejercicios

¿Qué pasa cuando comparás dos cosas en Python?

Python te responde con una de dos cosas: True o False.

Eso es todo. No hay más opciones.

¿5 es mayor que 3?

→

`5 > 3`

→

True

¿2 es mayor que 8?

→

`2 > 8`

→

False

¿10 es igual a 10?

→

`10 == 10`

→

True

`"Ana" == "ana"`

→

`"Ana" == "ana"`

→

False

Ojo — "Ana" y "ana" no son lo mismo. Python sí distingue mayúsculas.

Los 6 operadores — memorízalos, son siempre los mismos

No hay más. Con estos seis hacés todas las comparaciones.

==

¿Son iguales?

5 == 5 → True

!=

¿Son diferentes?

5 != 3 → True

>

¿El primero es mayor?

8 > 3 → True

<

¿El primero es menor?

2 < 7 → True

>=

¿Mayor o igual?

5 >= 5 → True

<=

¿Menor o igual?

3 <= 4 → True

El resultado lo podés guardar en una variable

Comparar devuelve True o False — y eso es un valor como cualquier otro. Lo podés guardar.

```
edad = 20

es_adulto = edad >= 18

print(es_adulto)    # True
```

¿Para qué sirve esto?

Si vas a usar esa comparación varias veces, la guardás una sola vez en vez de escribirla de nuevo cada vez.

```
# Repetir la comparación dos veces – feo y propenso a errores

if edad >= 18:
    print("Mayor de edad")

if edad >= 18:
    print("Puede votar")
```

```
# Guardarlo una vez – más limpio

es_adulto = edad >= 18

if es_adulto:
    print("Mayor de edad")

if es_adulto:
```

= y == no son lo mismo — este error lo van a cometer

Lo comete todo el mundo al principio. No se apaniquen cuando les pase, solo recuerden esto.

= asignar

Guarda algo en una variable.
No pregunta nada.

```
nombre = "Ana"

# Esto dice: guardá "Ana" en nombre
```

```
edad = 20      → guarda 20
nombre = "Ana" → guarda "Ana"
```

Un = solo siempre es asignar.

== comparar

Pregunta si dos cosas son iguales.
Devuelve True o False.

```
nombre == "Ana"

# ¿nombre vale "Ana"?
```

```
nombre == "Ana" → True o False
edad    == 20   → True o False
```

Dos == siempre es comparar.

Ejercicio 1

¿Qué imprime esto? Díganme antes de correrlo.

```
a = 10
b = 5

print(a > b)

print(a == 10)

print(b != 5)
```



Ahora este:

```
x = 7

resultado = x >= 10

print(resultado)
```

and, or, not — cuando una comparación no alcanza

Hasta ahora comparamos una cosa a la vez. Pero a veces la condición es más complicada que eso.

and

Los dos tienen que ser True

Si uno solo falla, ya es False.

```
edad >= 18 and tiene_id == True
```

→ *True solo si los dos se cumplen*

or

Con uno solo que sea True alcanza

Solo es False si los dos son False.

```
es_estudiante or es_adulto_mayor
```

→ *True si cualquiera de los dos es True*

not

Le da vuelta al resultado

True pasa a False. False pasa a True.

```
not True → False
```

```
not False → True
```

and — todos los casos posibles

Para que and dé True, los DOS tienen que ser True. Con que uno falle, ya es False.

condición A → True

and

condición B → True

→

True

*las dos son True
→ True*

condición A → True

and

condición B → False

→

False

*una es False →
False*

condición A → False

and

condición B → True

→

False

*una es False →
False*

condición A → False

and

condición B → False

→

False

*las dos son
False → False*

Regla fácil: con and, uno solo que falle arruina todo.

or — todos los casos posibles

or es lo contrario a and. Falla solo cuando los DOS son False. Si uno es True, ya está.

condición A → True

or

condición B → True

→

True

*los dos son True
→ obvio*

condición A → True

or

condición B → False

→

True

uno alcanza

condición A → False

or

condición B → True

→

True

uno alcanza

condición A → False

or

condición B → False

→

False

*ninguno →
False*

Regla fácil: con or, solo falla si todo falla.

not — el más simple de los tres

not agarra lo que sea y lo voltea. True → False, False → True. Nada más.

not True

→

False

True se vuelve False

not False

→

True

False se vuelve True

not (5 > 3)

→

False

5 > 3 es True, not lo voltea

not (2 > 8)

→

True

2 > 8 es False, not lo voltea

not (nombre == "Ana")

→

?

depende de qué valga nombre

¿Cuándo usarlo? Cuando es más claro decir 'que esto no pase' que describir todo lo que sí debe pasar.

Tabla de verdad — para consultar si se confunden

No la memoricen de golpe. Entiendan la lógica y ya la van a tener en la cabeza sola.

and — los dos tienen que ser True

A	B	A and B
✓ True	✓ True	✓ True
✓ True	✗ False	✗ False
✗ False	✓ True	✗ False
✗ False	✗ False	✗ False

or — basta con que uno sea True

A	B	A or B
✓ True	✓ True	✓ True
✓ True	✗ False	✓ True
✗ False	✓ True	✓ True
✗ False	✗ False	✗ False

Tres errores que van a ver seguido — propios y ajenos

No los digo para asustarlos. Los digo porque si los ven venir, los arreglan más rápido.

= en vez de ==

```
if nombre = "Ana":
```



```
if nombre == "Ana":
```

Python devuelve error de sintaxis. Un = adentro de un if siempre está mal.

Comparar texto con número sin convertir

```
edad = input("Edad: ")  
if edad >= 18:
```



```
edad = int(input("Edad: "))  
if edad >= 18:
```

input() devuelve texto. "20" no es 20. Sin int() Python no puede comparar.

not sin paréntesis cuando hay más de una condición

```
not a > 5 and b < 3
```



```
not (a > 5 and b < 3)
```

Sin paréntesis, not solo aplica a a > 5. Con paréntesis aplica a todo.

input() — el texto que parece número pero no lo es

Si el usuario escribe 20, Python guarda el texto "20". No el número 20. Son cosas distintas.

Para hacer cuentas o comparaciones con ese dato, hay que convertirlo antes con `int()` o `float()`.

✗ Sin convertir

```
edad = input("¿Cuántos años? ")  
  
# edad tiene el texto "20"  
  
# edad >= 18 → error o resultado raro
```

Python tiene "20". No puede comparar texto con números.

✓ Con int()

```
edad = int(input("¿Cuántos años? "))  
  
# int() convierte "20" al número 20  
  
# ahora edad >= 18 funciona bien
```

Ahora edad es el número 20 y todo funciona.

int(), float(), str() — convertir de un tipo a otro

Ya vieron int(). Los otros dos los van a necesitar también. Ninguno es complicado.

int()

desde str o float

```
int("20") → 20  
int(3.9) → 3
```

Corta los decimales, no redondea. int(3.9) da 3, no 4.

float()

desde str o int

```
float("3.14") → 3.14  
float(5) → 5.0
```

Agrega decimal aunque no lo necesite.

str()

desde int o float

```
str(42) → "42"  
str(3.14) → "3.14"
```

Útil para armar mensajes: "Tienes " + str(edad) + " años"

int("hola") da error. Solo funciona si lo que convertís realmente es un número.

Ejemplos reales — todo junto

¿Puede entrar al evento?

Condición: "mayor de 18 y tiene boleto"

```
edad = int(input("¿Cuántos años? "))  
  
tiene_boleto = True  
  
puede_entrar = edad >= 18 and tiene_boleto  
  
print(puede_entrar)
```

True si tiene las dos cosas

¿Tiene descuento?

Condición: "basta con ser estudiante o adulto mayor"

```
es_estudiante = True  
  
es_adulto_mayor = False  
  
tiene_descuento = es_estudiante or es_adulto_mayor  
  
print(tiene_descuento)
```

True — es_estudiante es True,
con eso alcanza

Ejercicio 2

Hagan un programa que haga esto:

1. Pida el nombre del usuario
2. Pida su edad (con `int()`)
3. Pida si tiene boleto — 1 para sí, 0 para no (con `int()`)
4. Imprima si puede entrar (tiene que ser mayor de 18 y tener boleto == 1)

Pistas:

1

`input()` tres veces — nombre sin `int()`, los otros dos con `int()`

2

`puede_entrar = edad >= 18 and boleto == 1`

3

`print("Hola", nombre, "— ¿puede entrar?", puede_entrar)`

Reto — si terminaron el ejercicio 2

Sistema de acceso con estas reglas:

- El usuario tiene que ser mayor de 18
- Tiene que poner la contraseña correcta (password == 1234)
- O puede ser administrador (admin == 1) — en ese caso se salta la contraseña

Impriman si se concedió el acceso o no.

```
puede_entrar = (edad >= 18 and password == 1234) or admin == 1
```

¿Dudas?

No hay preguntas tontas. Si algo no quedó claro, ahorita lo vemos.

La próxima clase: if, else, elif